

TEST PLAN DOCUMENT – DEMOBLAZE AUTOMATION TESTING

1. Test Plan ID

TP-DEMO-001

2. Introduction

Demo Blaze is a demo e-commerce web application that allows users to perform operations such as user registration, login, product browsing, product category selection, viewing product details, adding products to the cart, deleting products, and placing orders.

This Test Plan defines the **testing approach, scope, objectives, schedule, and deliverables** for functional and automation testing of the DemoBlaze application using **Selenium WebDriver, Java, and TestNG**.

3. Objectives

- Validate all core e-commerce functionalities.
 - Verify user Signup and Login functionality.
 - Verify Home page navigation and UI elements.
 - Validate product categories and product information.
 - Validate Add to Cart functionality.
 - Verify cart product details and price calculation.
 - Verify Delete product functionality.
 - Verify checkout and purchase functionality.
 - Validate blank and invalid checkout inputs.
 - Identify functional and UI defects.
 - Ensure stable and reliable automation execution.
-

4. Scope

In Scope

- User Signup functionality
- User Login, Logout functionality
- Home page validation, Navigation links
- Product category validation
 - Phones
 - Laptops
 - Monitors
- Product name, price, image, details validation
- Add to Cart functionality
- Cart product validation
- Delete product functionality
- Empty cart validation
- Place Order functionality
- Successful purchase validation
- Cart validation after purchase
- Checkout form validation
- Blank field validation during purchase
- Contact functionality
- About Us functionality
- Pagination (Next/Previous)
- UI element validation
- Defect identification and reporting

Out of Scope

- Backend/database validation
 - Advanced security testing
 - Performance/load testing
 - Real payment gateway validation
 - API testing
 - Mobile application testing
-

5. Features to be tested

User Management

- Signup
- Login/ Logout
- Valid and invalid credentials
- Blank field validation

Home Page

- Home page loading
- Page title
- Navigation links
- Product display
- Cart link
- Login link
- Signup link
- Contact link
- About Us link

Product Categories

- Phones category
- Laptops category
- Monitors category
- Category switching
- Products displayed under categories
- Product count
- Next/Previous pagination
- Product opening
- Scrolling

Product Details

- Product name
- Product price
- Product image
- Product description
- Image visibility
- Product name visibility
- Product name/image clickability
- Add to Cart functionality

Cart

- Open Cart
- Verify empty cart
- Add product to cart
- Verify product details
- Verify cart total
- Delete product
- Verify empty cart after deletion
- Multiple product validation
- Place Order with empty cart
- Successful purchase
- Verify cart after successful purchase

- Checkout form validation
- Blank field validation

Contact

- Contact link
- Contact modal
- Email field
- Name field
- Message field
- Send message
- Close button

About Us

- About Us link
 - About Us modal
 - Content validation
 - Close button
-

6. Features not to be Tested

- Database validation
 - Backend validation
 - Advanced security testing
 - Performance testing
 - Real payment processing
 - API testing
-

7. Test Deliverables

- Test Plan Document
 - Test Case Execution Report
 - Defect Report
 - Screenshots/Test Evidence
 - Final Project Report
 - Signoff Document
-

8. Entry Criteria

Testing will begin when:

- Application URL is accessible.
 - Test environment is ready.
 - Browser is installed and configured.
 - Java environment is configured.
 - Selenium WebDriver is configured.
 - TestNG framework is configured.
 - Maven project setup is completed.
 - Automation framework is ready.
 - Test data is available.
 - Test cases are prepared.
-

9. Exit Criteria

Testing will be considered complete when:

- All planned test cases are executed.
 - Core application functionalities are tested.
 - Automation execution is completed.
 - Critical and high-severity defects are identified and reported.
 - Failed test cases are analyzed.
 - Defects are documented with appropriate evidence.
 - Test execution summary is prepared.
 - Final project report is completed.
-

10. Test Environment

Component	Details
Application URL	https://www.demoblaze.com/
Application Type	Web Application
Browser	Google Chrome
OS	Windows 10+
Automation Tool	Selenium WebDriver
Framework	TestNG
Language	Java
Build Tool	Maven
Design Pattern	Page Object Model (POM)
Data Handling	Apache POI / Excel

Component	Details
IDE	Eclipse

11. Test Strategy

♦ Testing Types

- Functional Testing
- UI Testing
- Automation Testing
- Regression Testing
- Positive Testing
- Negative Testing
- Integration-level UI flow testing

♦ Approach

- Automation using **Page Object Model (POM)**
 - Test execution using **TestNG**
 - Assertions using TestNG assertions
 - Explicit waits for dynamic elements
 - Selenium WebDriver for browser automation
 - Apache POI for Excel-based test data where required
 - Screenshots used as test evidence
 - Defects documented separately from test execution results
-

12. Roles & Responsibilities

Role	Responsibility
QA Engineer	Design test cases, execute tests, validate application behavior and log defects
Automation Tester	Develop, execute and maintain Selenium automation scripts
Test Lead / QA Lead	Review test cases, execution results and defect reports
Developer	Analyze and fix reported defects
Project Manager	Monitor project progress and approve final deliverables

13. Test Schedule -30 Days

14. Risks and Mitigation

Risks

- DemoBlaze application may be temporarily unavailable.
- Dynamic elements may cause automation failures.
- UI changes may affect Selenium locators.
- Unexpected JavaScript alerts may interrupt execution.
- Network/server issues may cause test failures.
- Application defects may cause dependent test cases to fail.
- Browser/WebDriver compatibility issues may occur.
- Test data may already exist.

Mitigation

- Use appropriate explicit waits.
- Maintain reusable Page Object classes.
- Use stable locators wherever possible.
- Handle JavaScript alerts explicitly.
- Use proper test data management.
- Analyze failures before reporting them as application defects.
- Maintain compatible browser and Selenium versions.
- Re-execute failed tests when failures are caused by environmental issues.

21. Approval

Role	Name	Signature	Date
QA Engineer	_____	_____	_____
QA Lead	_____	_____	_____
Project Manager	_____	_____	_____
